

1 Motivation

A researcher’s taste—their ability to identify which problems are worth solving—is shaped by a massive, heterogeneous stream of information: reading papers, attending talks, conversations with collaborators, browsing Twitter, reviewing code. From all of this, the best researchers extract signal: they notice gaps between lines of work, judge which gaps matter, and connect ideas others haven’t linked. The result is a research direction that feels inevitable in retrospect but required consuming and synthesizing enormous context to identify.

Can I build models that do this? The vision is a language model that consumes massive streams of scientific information and asks the right questions—not just answers them. Standard pretraining learns $P(\text{paper} \mid \text{literature})$ —the distribution over published outcomes—but captures only the artifact. The intermediate reasoning—why an author read 40 papers, dismissed most, noticed a gap between two, and judged it worth three years of work—is invisible in the training data.

Recent work on synthetic pretraining suggests this is tractable. Yang et al. (2024) demonstrate that *knowledge rearrangement*—generating varied representations of the same facts through entity-graph traversal—boosts QA accuracy from 39.5% to 56.2% where naive continued pretraining actually *hurts* performance. Yang et al. (2025) scale this to general pretraining by modeling inter-document conditionals $P(d_2 \mid d_1)$, recovering 58% of a $20\times$ -data oracle at 1T tokens. Kim et al. (2025) show that *how* synthetic data is organized matters as much as its content: stitching related generations into long megadocuments with latent-thought rationales yields $1.80\times$ data efficiency, with benefits substantially larger on long-context evaluation. Dong et al. (2024) demonstrate that scientific evolution itself has learnable structure—chronological BERT on 1.75M arXiv papers achieves 0.991 AUC on citation prediction with smooth temporal weight trajectories.

The gap: these methods improve knowledge acquisition or predict citations, but none capture the intermediate reasoning of literature review. I want to synthesize *that process* as training data.

2 Approach: Citation Random Walks

I reconstruct research reasoning trajectories from arXiv and assemble them into long-context synthetic training data in three stages.

Stage 1: LLM-Guided Citation Crawl. I use the Kaggle arXiv metadata corpus for title→ID resolution and download per-paper LaTeX source from `arxiv.org/src/{id}`, extracting structured references via BibTeX/BBL parsing. The crawl is BFS over the citation graph: at each node, an LLM ranks references by importance with rationale, and I follow the top- K with arXiv source available. This produces a citation tree where edges carry explicit signals about *why* to follow each citation.

Stage 2: Synthetic Annotation. For each paper i in the tree, I run a sequence of LLM calls that build on each other with the full reading history as context. (1) *Summary* (~ 200 tokens): structured summary of the core problem, method, and results. (2) *Gap analysis* (~ 300 tokens): the LLM receives the summary *plus all prior summaries and gap analyses from papers* $1 \dots i-1$, identifying open questions and limitations that become visible only in the context of accumulated reading—gaps *between* papers, not just within them. (3) *Cross-paper connections* (every 2–3 papers, ~ 400 tokens): the LLM receives all annotations so far and generates explicit links—e.g., “Paper 3’s linear-time primitive could address Paper 1’s efficiency bottleneck, but needs Paper 2’s approximation guarantees.” (4) *Research idea* (final, ~ 500 tokens): a concrete problem statement, proposed method, and expected contribution. The ground truth is the actual paper at the root of the citation tree—the idea the author pursued after this literature review. All calls use a 70B+ instruction-tuned model via vLLM; total cost is ~ 1400 generated tokens per paper.

Stage 3: Long-Context Megadoc Assembly. Following Kim et al. (2025), I interleave paper content with annotations into long training sequences (Figure 1). Critically, I also mix in *irrelevant papers*—other arXiv papers published around the same time as the target paper, sampled from unrelated categories. This simulates reality: a researcher’s daily information stream includes many papers that have nothing to do with the idea they eventually pursue. The model must learn to selectively attend to what matters and ignore the rest, rather than treating every paper in the sequence as signal.

3 Training: TTT for Taste

Real information streams are long and noisy. There are two approaches to handling irrelevant context: (a) *retrieval*—filter the stream before the model sees it, or (b) *selective absorption*—let the model consume everything and learn what to attend to. I plan to test both, but TTT is specifically the answer to (b).

Paper 1 (full text or compressed): ‘‘Attention Is All You Need’’ --- Vaswani et al. 2017
<annot> Summary: introduces transformer... Gaps: quadratic attention cost, no recurrence...
<noise> ‘‘Protein Folding with Graph Networks’’ --- unrelated paper from same week
Paper 2: ‘‘Efficient Transformers: A Survey’’ --- Tay et al. 2022
<annot> Summary: categorizes efficient attention... Gaps: no unified theory of approximation...
Paper 3: ‘‘Learning to (Learn at Test Time)’’ --- Sun et al. 2024
<annot> Summary: TTT replaces fixed hidden state... Gaps: scaling to very long contexts...
<cross-paper> Papers 1+2: attention is powerful but costly; Paper 3 offers a different primitive...
<noise> ‘‘Reward Hacking in RLHF’’ --- unrelated paper from same month
Paper 4: ‘‘Mamba: Linear-Time Sequence Modeling’’ --- Gu & Dao 2023
<annot> Summary: selective SSM achieves... Gaps: no test-time adaptation, fixed representations...
<cross-paper> TTT (Paper 3) + SSM efficiency (Paper 4): linear-time <i>adaptive</i> sequence modeling?
<idea> <i>Target</i> : TTT layers with SSM-structured updates for efficient long-context adaptive inference.

Figure 1: Megadoc stitching. Papers (blue) interleaved with annotations (green), cross-paper reasoning (red), irrelevant papers (gray), and a target idea (gold). Noise papers simulate a real information stream.

Inner loop (test-time). The model processes the megadoc via gradient-based self-supervised learning, minimizing cross-entropy on content tokens. Its parameters *adapt* to the literature review trajectory, absorbing each paper and its annotations into its weights. Noise papers contribute less useful gradient signal; the model must learn to selectively update based on what will matter downstream.

Outer loop (meta-learning). After the inner loop processes the full sequence, I collect all accumulated annotations and present them alongside the adapted model’s representations. The outer loss is cross-entropy on the target research idea. The meta-gradient flows through the inner loop, training the model to absorb information during reading in a way that helps it generate good directions afterward. This directly addresses needle-in-a-haystack: the outer loop rewards inner-loop updates that selectively encode relevant information, even when buried among noise.

Practical constraints. I assume $\sim 1\text{M}$ arXiv papers ($\sim \text{half}$ the corpus), each markdown-converted to $\sim 15\text{K}$ tokens. A megadoc with 10 relevant papers, 5 noise papers, and annotations runs $\sim 250\text{K}$ tokens—well within the regime where TTT shows its advantage over fixed-context transformers. For the base model, I start from the pretrained TTT-3B checkpoint (Sun et al., 2024) and perform continued pretraining on the assembled megadocs. TTT-3B already implements the inner-loop self-supervised update; I am training it on data designed to reward selective absorption over long sequences. The annotation pipeline generates $\sim 1.4\text{B}$ synthetic tokens (at 1M papers), giving a total training corpus of $\sim 15\text{--}20\text{B}$ tokens.

4 Evaluation

I propose four evaluation strategies: (1) *Intrinsic*—CE on held-out research directions, plus auxiliary tasks (citation prediction, gap identification across years, importance ranking by follow-up work). (2) *LLM-as-Judge*—a strong LLM scores generated directions on novelty, specificity, grounding, feasibility. (3) *Generate $\rightarrow$ Execute $\rightarrow$ Judge*—model generates idea, code agent implements it, conference acceptance model scores the output as RLHF reward. (4) *Human expert*—blind comparison of generated vs. real research directions.

References

- Dong, J., Lyu, Z., & Ke, Q. (2024). Towards understanding evolution of science through language model series. *arXiv preprint arXiv:2409.09636*.
- Kim, K., Kotha, S., Choi, Y., Hashimoto, T., Haber, N., & Liang, P. (2025). Data-efficient pre-training by scaling synthetic megadocs. *arXiv preprint arXiv:2603.18534*.
- Sun, Y., Li, X., Dalvi, K., Xu, J., Vikram, A., Zhang, L., Dubois, Y., Chen, X., Wang, X., Kour, S., Bai, Y., Kolter, Z., & Hashimoto, T. (2024). Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*.
- Yang, Z., Band, N., Li, S., Candès, E., & Hashimoto, T. (2024). Synthetic continued pretraining. *arXiv preprint arXiv:2409.07431*.
- Yang, Z., Zhang, A., Liu, H., Hashimoto, T., Candès, E., Wang, C., & Pang, R. (2025). Synthetic bootstrapped pretraining. *arXiv preprint arXiv:2509.15248*.